COMPLETE JIRA NOTES

Beginner to Advanced • Practical Guide • Revision Handbook

SOFTWARE TESTING • PROJECT MANAGEMENT • AGILE • DEFECT TRACKING • JIRA ADMINISTRATION

A structured study and revision guide covering Jira fundamentals, projects, issues, Agile and Scrum, workflows, bug reporting, test management, JQL, dashboards, reports, administration, automation, integrations, and practical project flow.

Prepared as a self-study reference
Use the practical examples and quick-revision tables to prepare for Jira assignments, QA work, and interviews.

How to Use These Notes

Study the chapters in order the first time. For revision, use the quick-reference tables, JQL examples, bug life-cycle diagram, and practical checklist near the end. Jira menus and available features can vary by Jira edition, project type, permissions, and installed apps.

Contents

1. Introduction to Jira
2. Jira Fundamentals and Terminology
3. Jira Projects
4. Jira Issues and Issue Types
5. Agile Methodology and Scrum
6. Boards, Backlog, and Sprints
7. Workflow Management
8. Bug and Defect Management
9. Test Management in Jira
10. Issue Linking
11. Search, Filters, and JQL
12. Dashboards and Gadgets
13. Reports and Metrics
14. Jira Administration
15. Jira Automation
16. Jira Integrations
17. Real-Time Project Workflow
18. Practical Concepts and Common Mistakes
19. Interview Questions
20. Quick Revision Cheat Sheet
21. Practical Revision Checklist and Study Plan

PART 1 — Introduction to Jira

1. What is Jira?

Jira is a project management and issue-tracking tool developed by Atlassian. It is used by software development teams, testers, project managers, and business teams to plan, track, manage, and monitor work.

Jira organizes work into issues, assigns responsibilities, records progress, supports Agile practices, and provides reports and dashboards.

Example: E-commerce application

Imagine a company developing an online shopping application with registration, login, product search, cart, payment, and order tracking. A tester discovers that the application accepts an incorrect password. The tester records the defect in Jira, a developer investigates and fixes it, and the tester retests it. Jira keeps the issue, ownership, status, comments, evidence, and history together.

2. Why do teams use Jira?

Purpose	Explanation
Project management	Organize and monitor project activities.
Issue tracking	Track tasks, bugs, requirements, and improvements.
Bug tracking	Record, assign, investigate, and resolve defects.
Agile management	Manage Scrum and Kanban work.
Collaboration	Keep discussions and work history in one place.
Workflow management	Move work through defined stages.
Reporting	Monitor progress, workload, and trends.
Dashboards	Display key project information.

3. Jira products

Product	Purpose
Jira Software	Software development, issue tracking, and Agile project management.
Jira Service Management	IT service management and support workflows.
Jira Work Management / business capabilities	General business work management.
Jira Product Discovery	Capturing, prioritizing, and exploring product ideas.

4. Jira Cloud vs Data Center

Jira Cloud	Jira Data Center
Hosted and operated by Atlassian.	Hosted on infrastructure managed by the organization.
Accessed through the cloud.	Accessed through the organization's deployment.
Atlassian manages infrastructure and platform updates.	Organization manages infrastructure and upgrades.
Less infrastructure maintenance for customers.	More infrastructure administration and control.

PART 2 — Jira Fundamentals

5. Important Jira terminology

Term	Meaning
Project	Collection of related work.
Issue	An individual work item.
Epic	A large body of work that can be divided into smaller issues.
Story	A user requirement or functionality.
Task	A specific unit of work.
Bug	A defect in software.
Subtask	A smaller activity under a parent issue.
Sprint	A fixed-length period of work in Scrum.
Backlog	Prioritized list of work not yet completed.
Board	Visual representation of issues and their progress.
Workflow	Statuses and transitions an issue follows.
Assignee	Person responsible for an issue.
Reporter	Person who created or reported an issue.
Priority	How urgently work should be addressed.
Status	Current stage of an issue.
Resolution	Outcome recorded when an issue is completed or otherwise resolved.
Component	A functional area or module of a project.
Version	A product release or version.
Label	Keyword used to categorize issues.
Filter	Saved search for issues.
Dashboard	Visual summary using gadgets.

Four questions to remember: Issue = What work needs to be done? Assignee = Who is responsible? Status = What is the current stage? Priority = How urgently should it be handled?

6. Jira issue hierarchy

```
EPIC: E-Commerce Application
STORY: User Authentication
TASK: Develop Login Page
SUBTASK: Create Login Form
SUBTASK: Implement Validation
BUG: Login accepts incorrect password
STORY: Product Management
TASK: Implement Product Search
TASK: Implement Product Filtering
STORY: Order Management
TASK: Implement Order Tracking
```

An issue type is not automatically a parent-child relationship. For example, a bug can be related to a story through issue linking without being a child of that story. Exact hierarchy options depend on Jira configuration.

PART 3 — Jira Projects

7. What is a Jira project?

A project is a collection of issues associated with a product, application, team, or initiative. An e-commerce application might have one project containing issues for login, registration, product search, cart, checkout, payment, and order tracking.

8. Creating a Jira project

1. Open Jira and navigate to Projects.
2. Select Create project.
3. Choose a suitable template, such as Scrum or Kanban.
4. Select the project type if prompted.
5. Enter the project name and project key.
6. Review the settings and create the project.

Field	Example
Project name	First Web Site
Project key	SHOP
Project type	Software
Template	Scrum or Kanban

The project key is used in issue identifiers. Examples: SHOP-1, SHOP-2, SHOP-3. Each issue has a unique key within the Jira site.

9. Scrum project vs Kanban project

Scrum	Kanban
Work is planned in sprints.	Work flows continuously.
Uses sprint planning and sprint goals.	No mandatory sprint planning.
Sprint backlog is central.	Kanban board and flow are central.
Useful for iterative product development.	Useful for support and continuous delivery.

PART 4 — Jira Issues

10. What is an issue?

An issue is one unit of work in Jira. It can represent a requirement, task, defect, test activity, technical improvement, or documentation activity. Example issue key: SHOP-5.

11. Common issue types

Issue type	Purpose	Example
Epic	Large body of work.	User Authentication.
Story	User requirement or functionality.	As a customer, I want to log in securely.
Task	Specific work to complete.	Develop login page.
Bug	Software defect.	Incorrect password is accepted.
Subtask	Smaller activity under a parent issue.	Implement password validation.

User story format

```
As a [user], I want [functionality], so that [benefit].
```

12. Creating an issue

1. Open the Jira project and select Create.
2. Select the project and issue type.
3. Enter a concise summary and a useful description.
4. Set priority and assignee where appropriate.
5. Add labels, components, versions, or due date if needed.
6. Attach screenshots, logs, or other evidence where useful.
7. Create the issue and verify the saved details.

13. Important issue fields

Field	Explanation
Project	Project to which the issue belongs.
Issue Type	Bug, Task, Story, Epic, and so on.
Summary	Short title describing the issue.
Description	Detailed explanation of the work or defect.
Assignee	Person responsible.
Reporter	Person who created the issue.
Priority	Urgency or importance.
Status	Current stage.
Labels	Keywords for classification.
Components	Application modules or functional areas.
Fix Version	Release in which the issue is fixed or planned to be fixed.
Affects Version	Release in which the defect occurs.
Due Date	Expected completion date.
Attachment	Screenshot, log, or document.
Linked Issues	Related Jira issues.

Summary vs description

Summary: A short title, for example, “Login accepts an incorrect password.”

Description: Reproducible details, preconditions, steps, expected result, and actual result.

```
Preconditions:
A registered customer account exists.

Steps to Reproduce:
1. Open the shopping application.
2. Navigate to the login page.
3. Enter a registered email address.
4. Enter an incorrect password.
5. Click Login.

Expected Result:
The application rejects the login and displays an appropriate error.

Actual Result:
The application allows login despite the incorrect password.
```

PART 5 — Agile Methodology and Scrum

14. What is Agile?

Agile is an iterative and incremental approach to software development. Work is divided into smaller pieces that are developed, tested, reviewed, and improved. Agile encourages frequent delivery, collaboration, feedback, and adaptation to change.

Example

- Iteration 1: Registration and login.
- Iteration 2: Product management.
- Iteration 3: Shopping cart.
- Iteration 4: Payment integration.
- Iteration 5: Order tracking.

15. Agile principles — summary

1. Satisfy customers through early and continuous delivery.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently.
4. Business people and developers collaborate regularly.
5. Build projects around motivated individuals.
6. Prefer direct communication.
7. Working software is the primary measure of progress.
8. Maintain a sustainable pace.
9. Pay continuous attention to technical excellence and good design.
10. Keep solutions simple.
11. Encourage self-organizing teams.
12. Regularly reflect and adjust how the team works.

16. Scrum framework

Scrum is an Agile framework for managing complex work through short, fixed-length iterations called sprints.

Scrum accountabilities

Accountability	Responsibilities
Product Owner	Maximizes product value, manages and orders the Product Backlog, clarifies product goals and requirements.
Scrum Master	Helps establish Scrum, facilitates effectiveness, and helps address impediments.
Developers	Create a usable increment each sprint and plan and perform the work needed to meet the Sprint Goal.

17. Scrum events

Event	Purpose
Sprint	Container for all Scrum events; fixed length, usually one month or less.
Sprint Planning	Agree why the sprint is valuable, what can be done, and how work will be approached.
Daily Scrum	Inspect progress toward the Sprint Goal and adapt the plan.
Sprint Review	Inspect the outcome with stakeholders and consider future adaptations.
Sprint Retrospective	Plan ways to improve quality and effectiveness.

18. Scrum artifacts

Artifact	Meaning
Product Backlog	Emergent, ordered list of what is needed to improve the product.
Sprint Backlog	Sprint Goal, selected Product Backlog items, and the plan for delivering them.
Increment	Usable, completed product work that meets the Definition of Done.

Product Backlog = all ordered product work. Sprint Backlog = selected sprint work plus the plan. Increment = usable completed product work.

PART 6 — Boards, Backlog, and Sprints

19. What is a Jira board?

A board is a visual way to view and manage issues. It typically displays issues as cards in columns mapped to workflow statuses.

```
TO DO IN PROGRESS DONE
Create login Develop login Create database
Test search Fix payment bug Design homepage
```

20. Scrum board and Kanban board

Scrum board	Kanban board
Supports sprint planning and sprint execution.	Supports continuous flow of work.
Commonly used with a backlog and active sprint.	Commonly uses columns and work-in-progress limits.
Tracks work toward a Sprint Goal.	Tracks flow and delivery through workflow stages.

21. What is a backlog?

A backlog is an ordered list of work that may be selected for development. The Product Owner is accountable for effective Product Backlog management.

```
Product Backlog
1. User Registration
2. User Login
3. Forgot Password
4. Product Search
5. Shopping Cart
6. Payment Integration
7. Order Tracking
```

22. What is a sprint?

A sprint is a fixed-length period of one month or less in which the team works toward a Sprint Goal and creates a usable increment.

Example: two-week sprint

```
Day 1 Sprint Planning
Days 2–13 Development and testing
Daily Daily Scrum
Day 14 Sprint Review
Day 14 Sprint Retrospective
```

Creating a sprint in Jira

1. Open a Scrum project's Backlog.
2. Select Create Sprint.
3. Move or select backlog issues for the sprint.
4. Open sprint planning and enter a sprint name, dates, and Sprint Goal.
5. Check team capacity and whether the selected work is realistic.
6. Start the sprint.

Completing a sprint

1. Open the active sprint and review issue statuses.
2. Confirm which work meets the Definition of Done.
3. Complete the sprint using Jira's sprint action.
4. Move incomplete work back to the backlog or another sprint as appropriate.
5. Review the sprint outcome and discuss improvements.

A sprint can be completed even if some issues remain incomplete. Incomplete work is not automatically considered Done; it must be replanned.

PART 7 — Jira Workflow Management

23. What is a workflow?

A workflow defines the statuses and transitions an issue follows from creation through completion. In short: workflow = statuses + transitions + rules.

```
TO DO
|
| Start Progress
v
IN PROGRESS
|
```

```
| Submit for Testing
v
TESTING
|
| Pass
v
DONE
```

24. What is a status?

Status	Meaning
Open	Issue has been created or is awaiting work.
To Do	Work has not started.
In Progress	Work is being performed.
In Review	Work is awaiting review.
Testing	Work is being tested.
Resolved	A resolution has been provided.
Done	Work is considered complete.
Reopened	Issue needs further work after being reopened.

25. What is a transition?

A transition is an action that moves an issue from one status to another. For example, “Start Progress” may move an issue from To Do to In Progress.

Status tells where the issue is. Transition tells how it moves.

26. Workflow rules: conditions, validators, post functions

Element	Purpose	Example
Condition	Controls whether a transition is available.	Only members of a particular project role can use a transition.
Validator	Checks required data before a transition completes.	Require a resolution or a field value.
Post function	Performs an action after a transition.	Set a field or add an entry to issue history.

27. Creating and associating a workflow

1. Open Jira settings and locate Workflows; exact navigation depends on Jira version and permissions.
2. Create or edit a workflow.
3. Add the required statuses and transitions.
4. Configure conditions, validators, and post functions only where needed.
5. Publish or save the workflow.
6. Associate it with issue types through a workflow scheme if required.
7. Test the workflow using representative issues and permissions.

Example bug workflow

```
OPEN → ASSIGNED → IN PROGRESS → FIXED → RETEST
| |
PASS FAIL
| |
v v
CLOSED REOPENED
|
v
IN PROGRESS
```

A real organization may use different statuses and transition names. A workflow should reflect the team's actual process rather than adding unnecessary stages.

PART 8 — Bug and Defect Management

28. What is a bug?

A bug is a commonly used term for a software defect: a flaw that can cause actual behavior to differ from expected behavior. Example: a login form accepts an incorrect password.

29. Error, defect, bug, and failure

Term	Meaning
Error	A human mistake.
Defect	A flaw in a software work product.
Bug	Common informal term for a software defect.
Failure	Incorrect behavior observed when the software executes.

30. Bug life cycle

```
NEW
|
v
ASSIGNED
|
v
IN PROGRESS
|
v
FIXED
|
v
RETEST
| \
PASS FAIL
| |
v v
CLOSED REOPENED
|
v
IN PROGRESS
```

Other possible outcomes include Rejected, Duplicate, Cannot Reproduce, Deferred, and Won't Fix. These are not mandatory statuses; the actual workflow depends on the organization.

31. Bug status explanations

Status	Explanation
New	Tester has reported the defect.
Open	Issue is accepted for investigation.
Assigned	Issue has an owner.
In Progress	Developer is investigating or fixing it.
Fixed	Developer indicates a fix has been implemented.
Retest	Tester verifies the fix.
Closed	Tester or workflow confirms the issue is resolved.
Reopened	Defect still exists or the fix is insufficient.
Rejected	Reported behavior is not accepted as a defect.
Duplicate	Same issue has already been reported.
Deferred	Fix is postponed.
Cannot Reproduce	Team cannot reproduce the reported behavior.

32. Severity vs Priority

Severity describes the impact of a defect on the system or users. **Priority** describes how urgently the issue should be addressed. Teams may use different scales and may assign these fields through a triage process.

Severity	Typical impact	Example
Critical	Catastrophic or extremely serious impact.	Major data corruption or a critical security compromise.
High	Major functionality is broken.	Login or payment unavailable.
Medium	Functionality is affected, but a workaround may exist.	Product filter does not work.
Low	Minor impact.	Small text alignment issue.

Priority	Meaning
Highest / Critical	Must be addressed immediately.
High	Should be addressed soon.
Medium	Can be handled in planned work.
Low	Can be handled later.

Examples

- High severity, high priority: payment functionality is completely broken.
- Low severity, high priority: company logo is incorrect immediately before a major launch.
- High severity, low priority: a serious issue exists in a legacy feature scheduled for retirement, subject to risk review.
- Low severity, low priority: minor alignment issue on a rarely visited page.

Severity = how serious is the impact? Priority = how soon should the team fix it?

33. How to report a bug professionally

A useful bug report enables another person to reproduce, investigate, and verify the defect. Include the following information where applicable:

- Clear summary and description.
- Preconditions and test data.
- Numbered steps to reproduce.
- Expected result and actual result.
- Severity and priority, according to team practice.
- Environment: build, browser, operating system, device, and test environment.
- Attachments such as screenshots, recordings, logs, or network traces.
- Reporter, assignee, and relevant linked issues.

Professional bug report example

SHOP-5 — Login accepts an incorrect password

Issue Type: Bug
Preconditions: A registered customer account exists.
Steps: 1. Open the shopping application. 2. Navigate to the login page. 3. Enter the registered email address. 4. Enter an incorrect password. 5. Click Login.
Expected Result: The application rejects the login and displays an appropriate error message.
Actual Result: The application allows login despite the incorrect password.
Environment: Chrome, Windows 11, test environment.

Severity: Assess based on verified impact and security implications. **Priority:** Set through team triage. A suspected authentication bypass should be investigated urgently; do not assume a severity value without confirming the behavior and impact.

34. What happens after a bug is reported?

1. Tester discovers and reproduces the defect.
2. Tester creates a Jira bug with clear steps and evidence.
3. The team reviews or triages the report.
4. The issue is assigned to an appropriate owner.
5. Developer investigates and implements a fix.
6. Developer moves the issue to a state such as Fixed, according to workflow.
7. Tester retests the fix in the relevant build or environment.
8. If successful, the issue is closed; if it still fails, it is reopened with updated evidence.

In normal practice, the tester reports and verifies the defect; the developer investigates and fixes it. Follow the team's ownership and workflow rules.

PART 9 — Test Management in Jira

35. What is test management?

Test management is the planning, organization, execution, monitoring, and reporting of software testing. Jira can track testing work and defects. Jira Software by itself does not necessarily provide a full dedicated test-case management system; apps such as Xray or Zephyr can add test cases, test plans, and test execution capabilities.

36. Important testing terminology

Term	Meaning
Test Scenario	High-level functionality or situation to test.
Test Case	Detailed steps, data, and expected results for a particular check.
Test Data	Input values used during testing.
Preconditions	Conditions that must exist before a test.
Expected Result	Correct behavior the application should show.
Actual Result	Behavior observed during execution.
Test Execution	Running a test case and recording its result.
Test Plan	Organized approach and scope for testing.
Test Suite	Group of related test cases.
Test Report	Summary of test execution and outcomes.

37. Test scenario vs test case

Test Scenario: Verify the login functionality. This high-level scenario may contain several test cases:

1. Login with valid credentials.
2. Login with an invalid password.
3. Login with an invalid email.
4. Submit with empty fields.
5. Login with an unregistered email.
6. Verify password masking.

Test Scenario	Test Case
High-level testing objective.	Detailed procedure to verify a specific condition.
Broad description.	Specific steps, data, and expected result.
May contain multiple test cases.	Usually focuses on one particular behavior.

38. Writing a test case

Field	Example
Test Case ID	TC_LOGIN_001
Title	Verify login with valid credentials
Preconditions	Registered account exists.
Test Data	Valid email and password.
Steps	Enter credentials and click Login.
Expected Result	User should successfully log in.
Actual Result	Fill after execution.
Status	Not Executed before running.

39. Test execution results

Result	Meaning
Pass	Actual result matches expected result.
Fail	Actual result differs from expected result.
Blocked	Test cannot proceed because of a dependency or environment issue.
Not Executed	Test has not yet been run.
Not Applicable	Test does not apply to the current scope.

40. Relationship between test cases and bugs

```
Test Scenario: Login Functionality

TC_LOGIN_001: Valid login → PASS
TC_LOGIN_002: Invalid password → FAIL → SHOP-5 bug
TC_LOGIN_003: Empty password → PASS
```

When a test fails, investigate whether the failure is caused by a product defect, test-data issue, environment problem, or an incorrect test. Link the failed test to the defect when the Jira configuration or test-management app supports it.

PART 10 — Issue Linking

41. What is issue linking?

Issue linking connects Jira issues to show relationships and dependencies. Examples include a bug related to a story, one issue blocking another, or two issues being duplicates.

Link type	Meaning
blocks	This issue prevents another issue from progressing.
is blocked by	This issue cannot proceed because another issue is pending.
relates to	The issues are associated.
duplicates	This issue duplicates another.
is duplicated by	Another issue duplicates this one.
causes	This issue causes another issue.
is caused by	This issue is caused by another.

```
SHOP-2: User Login Story
|
| relates to
|
SHOP-5: Login accepts incorrect password
```

How to link issues

1. Open the issue.
2. Open the issue actions or More menu.
3. Select Link or Link issue.
4. Choose the relationship type.
5. Search for and select the other issue.
6. Confirm and verify the link.

Issue linking records a relationship. It does not automatically make the issues share the same status or workflow.

PART 11 — Search, Filters, and JQL

42. What are Jira filters?

A filter is a saved search that retrieves issues matching selected conditions. For example, a tester can save a filter showing all unresolved bugs in the SHOP project.

43. Basic search and advanced search

Basic search provides a visual interface for selecting fields and conditions. Advanced search uses JQL (Jira Query Language) for more flexible queries.

44. What is JQL?

JQL stands for **Jira Query Language**. It is used to search and filter Jira issues. It is similar in concept to SQL because it expresses structured conditions, but JQL is designed for Jira issues rather than relational database tables.

Basic JQL examples

All issues in a project

```
project = SHOP
```

All bugs

```
project = SHOP AND issuetype = Bug
```

All open issues

```
project = SHOP AND status = Open
```

High-priority bugs

```
project = SHOP AND issuetype = Bug AND priority = High
```

Issues assigned to current user

```
assignee = currentUser()
```

Unresolved bugs

```
project = SHOP AND issuetype = Bug AND resolution IS EMPTY
```

Issues reported by current user

```
reporter = currentUser()
```

Issues in progress

```
project = SHOP AND status = "In Progress"
```

Recently created issues

```
project = SHOP ORDER BY created DESC
```

Issues assigned to a particular user

```
assignee = "username"
```

High or highest priority bugs

```
project = SHOP AND issuetype = Bug AND priority IN (High, Highest)
```

Unresolved issues

```
project = SHOP AND resolution IS EMPTY
```

Issues due in the next seven days

```
project = SHOP AND due >= startOfDay() AND due <= endOfDay("+7d")
```

Issues updated in the last seven days

```
project = SHOP AND updated >= -7d
```

Summary contains login

```
project = SHOP AND summary ~ "login"
```

45. Important JQL operators

Operator	Purpose
=	Equal to.
!=	Not equal to.
IN	Matches one of multiple values.
NOT IN	Excludes specified values.
IS EMPTY	Field has no value.
IS NOT EMPTY	Field has a value.
AND	Both conditions must be true.
OR	Either condition can be true.
ORDER BY	Sorts results.
~	Text search.
!~	Text does not contain.

46. Saving a filter

1. Open issue search.
2. Enter a JQL query or build a basic search.
3. Run the search and check the results.
4. Select Save As.
5. Give the filter a clear name, such as “SHOP – High Priority Bugs”.
6. Configure sharing permissions if appropriate.
7. Use the saved filter in dashboards or reports where supported.

JQL field names and supported functions can vary with Jira configuration and edition. If a query returns an error, use Jira's autocomplete and field suggestions.

PART 12 — Jira Dashboards

47. What is a Jira dashboard?

A dashboard is a visual summary of project information. It can display issue lists, charts, activity, and sprint information using gadgets.

48. What is a gadget?

A gadget is an individual dashboard component. Common gadgets include:

Gadget	Purpose
Filter Results	Displays issues from a saved filter.

Gadget	Purpose
Pie Chart	Shows issue distribution by a selected field.
Created vs Resolved Chart	Compares issues created and resolved over time.
Issue Statistics	Groups issue counts by a selected field.
Two Dimensional Filter Statistics	Shows issue counts across two fields.
Average Age Chart	Shows the average age of unresolved issues.
Activity Stream	Shows recent project activity.
Assigned to Me	Shows issues assigned to the current user.
Sprint Burndown	Displays remaining sprint work.
Sprint Health	Shows sprint progress and related information.
Roadmap	Shows planned work and releases where available.

49. Adding and configuring dashboard gadgets

1. Open Dashboards and select or create a dashboard.
2. Select Add Gadget.
3. Search for and add the required gadget.
4. Open its configuration.
5. Select the saved filter or project data source.
6. Choose fields, columns, statistics, and refresh settings as needed.
7. Save and arrange gadgets on the dashboard.

Example: Pie Chart for bug priority

Setting	Value
Saved Filter	SHOP – All Bugs
Statistic Type	Priority
Refresh	As required

Example: Filter Results for unresolved bugs

```
project = SHOP AND issuetype = Bug AND resolution IS EMPTY
```

Possible columns: Key, Summary, Priority, Status, and Assignee.

Created vs Resolved Chart

Compares the number of issues created with the number resolved over a selected period. It can help a team understand whether its unresolved workload is increasing or decreasing.

50. Dashboard vs board

Dashboard	Board
Displays project information and metrics.	Displays issues and workflow progress.
Uses gadgets.	Uses columns and issue cards.
Focuses on monitoring and reporting.	Focuses on managing day-to-day work.
Can combine multiple saved filters.	Displays issues matching its board filter.

PART 13 — Jira Reports and Metrics

51. Burndown chart

A burndown chart shows how much work remains during a sprint. It helps the team inspect progress toward completing the Sprint Goal.

52. Burnup chart

A burnup chart shows completed work against total scope. It can make scope changes easier to see.

Burndown = remaining work. Burnup = completed work compared with total scope.

53. Velocity chart

A velocity chart shows how much work a Scrum team completes across sprints, commonly measured in story points. It is useful for forecasting, not for comparing individual productivity.

Sprint	Completed story points
Sprint 1	20
Sprint 2	25
Sprint 3	23
Sprint 4	28
Average	24

Average velocity = (20 + 25 + 23 + 28) ÷ 4 = **24 story points per sprint**.

54. Sprint report

A sprint report summarizes completed work, incomplete work, and scope changes during a sprint.

55. Control chart

A control chart shows how long issues take to move through a workflow. It helps teams understand cycle time and identify variation or bottlenecks.

56. Cumulative Flow Diagram

A cumulative flow diagram displays the number of issues in different workflow stages over time. It helps identify bottlenecks and excessive work in progress.

PART 14 — Jira Administration

57. What is Jira administration?

Jira administration involves configuring and maintaining users, permissions, projects, issue types, workflows, fields, screens, and other settings. Some settings require site, organization, or project administrator permissions.

58. User management

Users are people who access Jira. Administrators may manage accounts, groups, product access, and project roles. Example groups include Developers, Testers, Project Managers, and Administrators.

59. Permissions

Permission	Purpose
Browse Projects	View project issues.
Create Issues	Create new issues.
Edit Issues	Modify issue details.
Delete Issues	Delete issues, where permitted.
Assign Issues	Assign issues.
Transition Issues	Move issues through workflow transitions.
Resolve Issues	Resolve issues.
Manage Sprints	Manage sprint activities.

Permission scheme: Defines which users, groups, or project roles have particular permissions in a project.

60. Notification schemes

Notification schemes determine who receives notifications for events such as issue creation, assignment, updates, resolution, and comments.

61. Issue type schemes

An issue type scheme controls which issue types are available in a project. Example: Epic, Story, Task, Bug, and Subtask.

62. Workflow schemes

A workflow scheme associates workflows with issue types. For example, a project might use a development workflow for stories and tasks and a separate bug workflow for bugs.

Issue Type	Example workflow
Story	Development Workflow
Task	Development Workflow
Bug	Bug Workflow
Epic	Epic Workflow

63. Field configuration

Field configuration controls whether fields are required, optional, or hidden. Example: an organization may require a Description when creating a bug.

64. Screens

Screens control which fields appear during issue operations.

Screen	Purpose
Create screen	Fields shown when creating an issue.
Edit screen	Fields shown when editing an issue.
View screen	Fields shown when viewing an issue.

PART 15 — Jira Automation

65. What is Jira automation?

Jira automation executes predefined actions when triggers and conditions are met. A typical rule contains: **Trigger + Condition + Action**.

Example 1: Automatically assign a new bug

```
TRIGGER: Issue created
CONDITION: Issue type = Bug
ACTION: Assign issue to QA lead
```

Example 2: Notify a developer

```
TRIGGER: Issue assigned
ACTION: Send notification to the assignee
```

Example 3: Transition an approved issue

```
TRIGGER: Review field changes
CONDITION: Review status = Approved
ACTION: Transition issue to the configured next status
```

Example 4: Notify about overdue issues

```
TRIGGER: Scheduled
CONDITION: Due date is before today
AND resolution is empty
ACTION: Send notification
```

66. Benefits of automation

- Saves time and reduces repetitive work.
- Improves consistency and reduces manual errors.
- Speeds up issue processing.
- Provides automatic notifications.
- Supports standardized workflows.

PART 16 — Jira Integrations

67. What is Jira integration?

Integration connects Jira with other tools so teams can coordinate work across development, testing, documentation, and communication.

Tool	Purpose
GitHub	Source code management and development activity.
Bitbucket	Source code management.
Jenkins	Continuous integration and delivery automation.
Confluence	Documentation and knowledge sharing.
Slack	Team communication.
Microsoft Teams	Team collaboration.
Xray	Test management.
Zephyr	Test management.

Example: GitHub integration

A developer can include a Jira issue key in a commit message so the development activity can be associated with the issue when the integration is configured.

```
SHOP-5 Fix incorrect password validation
```

PART 17 — Real-Time Jira Project Workflow

68. Complete e-commerce project example

Project: First Web Site. Project key: SHOP. The example below connects project setup, requirements, development, testing, defect reporting, and reporting.

1. Create a Jira software project named First Web Site with key SHOP.
2. Create an Epic named User Authentication.
3. Create stories: SHOP-1 User Registration, SHOP-2 User Login, and SHOP-3 Forgot Password.
4. Create implementation tasks such as designing the registration form, developing login functionality, and implementing password validation.
5. Create a sprint named Authentication Sprint and select appropriate backlog items.
6. Assign development work to the responsible team members.
7. Developers move work through the configured workflow, for example To Do → In Progress → Review → Done.
8. Testers prepare and execute test cases for the stories.
9. Record test results. For example, valid login passes and invalid-password login fails.
10. Create a bug: SHOP-5 — Login accepts an incorrect password.
11. Review and assign the bug to the appropriate developer or triage owner.
12. Developer investigates, fixes the defect, and updates the issue according to the workflow.
13. Tester retests the fix in the relevant build and environment.
14. If the fix works, close the bug; otherwise reopen it with evidence.
15. Link SHOP-5 to the User Login story.
16. Create saved filters, dashboards, and reports to monitor the work.
17. Complete the sprint, review unfinished work, and discuss improvements.

69. End-to-end flow

```
REQUIREMENTS
|
v
TEST PLANNING
|
v
TEST CASE DESIGN
|
v
TEST EXECUTION
|
+------ PASS ------> RECORD PASS
|
+------ FAIL ------> INVESTIGATE
|
v
REPORT BUG
|
v
DEVELOPER FIX
|
v
RETEST
/ \
PASS FAIL
| |
v v
CLOSE REOPEN
|
v
REPORT AND MONITOR
```

PART 18 — Practical Concepts and Common Mistakes

70. Resolution vs Status

Status is the current stage of an issue. **Resolution** records the outcome when an issue is considered resolved or completed.

Status example	Possible resolution
Open	Unresolved
In Progress	Unresolved
Resolved	Fixed
Closed	Fixed
Closed	Won't Fix
Closed	Duplicate

A workflow should set and clear resolution consistently. An issue shown in a completed status but still unresolved can cause misleading filters and reports.

71. Assignee vs Reporter

Assignee	Reporter
Person responsible for working on the issue.	Person who created or reported the issue.
May change during the issue life cycle.	Usually remains the original reporter.

Example: Hari reports a bug and a developer is assigned. Reporter = Hari; Assignee = developer.

72. Component vs Label

Component	Label
Represents a project/module area.	Represents a keyword or tag.
Usually configured at project level.	Can be added to issues as needed.
Example: Authentication.	Examples: login, security, regression.

73. Fix Version vs Affects Version

Affects Version: Version in which the bug occurs. **Fix Version:** Version in which the bug is planned to be fixed or has been fixed.

Field	Example
Affects Version	1.0
Fix Version	1.1

74. Common Jira mistakes

1. Creating duplicate bugs without searching existing issues.
2. Writing vague issue summaries.
3. Leaving out reproducible steps.
4. Confusing severity with priority.
5. Assigning issues without confirming ownership.
6. Choosing an incorrect issue type.
7. Forgetting to link related issues when useful.
8. Leaving issues in incorrect workflow statuses.
9. Failing to set or clear resolution correctly.
10. Creating dashboards without saved filters or appropriate permissions.
11. Starting a sprint without reviewing its backlog and goal.
12. Closing a bug without retesting the fix.
13. Combining multiple unrelated defects into one bug.
14. Failing to attach useful evidence.

PART 19 — Jira Interview Questions and Answers

1. What is Jira?

Jira is an Atlassian project management and issue-tracking tool used to manage tasks, bugs, requirements, and Agile work.

2. What is an issue?

An individual unit of work, such as a task, bug, story, or epic.

3. What is a project?

A collection of related issues organized around a product, team, or initiative.

4. What is a Jira workflow?

The statuses and transitions an issue follows during its life cycle.

5. What is a Jira board?

A visual interface for managing and tracking issues.

6. What is a sprint?

A fixed-length period in Scrum during which the team works toward a Sprint Goal.

7. What is a backlog?

An ordered list of work that may be selected for development.

8. What is JQL?

Jira Query Language, used to search and filter Jira issues.

9. Difference between task and story?

A story describes a user need; a task describes a specific piece of work.

10. Difference between severity and priority?

Severity describes impact; priority describes urgency.

11. What is the bug life cycle?

The stages a defect passes through from reporting to closure or another outcome.

12. What is issue linking?

A way to record relationships and dependencies between Jira issues.

13. What is a dashboard?

A visual summary of project information using gadgets.

14. What is a filter?

A saved search for issues matching specified conditions.

15. Difference between Scrum and Kanban?

Scrum organizes work into fixed-length sprints; Kanban manages continuous flow.

16. What is a workflow scheme?

A configuration that associates workflows with issue types.

17. What is a permission scheme?

A configuration that defines which users or groups can perform actions in a project.

18. What is a notification scheme?

A configuration that defines who receives notifications for issue events.

19. What is a validator?

A rule that checks required information before a workflow transition completes.

20. What is a post function?

An action performed after a workflow transition.

21. What is Jira automation?

Rules that perform actions based on triggers and conditions.

22. What is a burndown chart?

A chart showing remaining work during a sprint.

23. What is a velocity chart?

A chart showing completed work across sprints.

24. Difference between resolution and status?

Status is the current stage; resolution is the recorded outcome.

25. Can Jira be used for test management?

Jira can track testing tasks and bugs. Apps such as Xray and Zephyr add dedicated test management features.

PART 20 — Quick Revision Cheat Sheet

75. One-line definitions

Concept	Remember
Jira	Project management and issue-tracking tool.
Project	Collection of related work.
Issue	Individual work item.
Epic	Large body of work.
Story	User requirement.
Task	Specific activity.
Bug	Software defect.
Subtask	Smaller activity under a parent issue.
Backlog	Ordered list of pending work.
Sprint	Fixed-length Scrum iteration.
Board	Visual work-tracking interface.
Workflow	Statuses and transitions.
Status	Current stage.
Transition	Movement between statuses.
Assignee	Responsible person.
Reporter	Issue creator.
Priority	Urgency.
Severity	Impact.
Filter	Saved issue search.
JQL	Jira Query Language.
Dashboard	Visual project summary.
Gadget	Dashboard component.
Automation	Rule-based automatic action.
Resolution	Outcome of an issue.

76. Important differences

Comparison	Remember
Epic vs Story	Large body of work vs user requirement.
Story vs Task	User need vs work activity.
Task vs Subtask	Main activity vs smaller activity.
Bug vs Task	Defect vs planned work.
Severity vs Priority	Impact vs urgency.
Status vs Transition	Current stage vs movement.
Assignee vs Reporter	Responsible person vs creator.
Scrum vs Kanban	Sprint-based vs continuous flow.
Product Backlog vs Sprint Backlog	All ordered product work vs selected sprint work and plan.
Board vs Dashboard	Work management vs monitoring.
Resolution vs Status	Outcome vs current stage.
Component vs Label	Project module vs keyword.
Fix Version vs Affects Version	Release with fix vs release affected.

77. JQL commands to memorize

Requirement	JQL
All project issues	project = SHOP
All bugs	issuetype = Bug
All tasks	issuetype = Task
All stories	issuetype = Story
Open issues	status = Open
In-progress issues	status = "In Progress"
High-priority bugs	issuetype = Bug AND priority = High
My issues	assignee = currentUser()
My reported issues	reporter = currentUser()
Unresolved issues	resolution IS EMPTY
Recently created	ORDER BY created DESC

78. Bug life cycle — memorize

```
NEW → ASSIGNED → IN PROGRESS → FIXED → RETEST
|
+---------+---------+
| |
PASS FAIL
| |
CLOSED REOPENED
```

79. Typical workflow — memorize

```
TO DO → IN PROGRESS → IN REVIEW → TESTING → DONE
```

PART 21 — Practical Revision Checklist and Study Plan

80. Practical Jira checklist

Jira fundamentals

- [ ] Explain what Jira is and why it is used.
- [ ] Explain projects and project keys.
- [ ] Understand issue types and hierarchy.
- [ ] Create a project and navigate its settings.

Issue management

- [ ] Create an Epic, Story, Task, and Bug.
- [ ] Edit issue fields and descriptions.
- [ ] Assign issues and update priorities.
- [ ] Create subtasks.
- [ ] Add attachments and comments.

Agile and Scrum

- [ ] Explain Agile and Scrum.
- [ ] Understand Scrum accountabilities and events.
- [ ] Create and manage a backlog.
- [ ] Create, start, and complete a sprint.
- [ ] Understand Scrum and Kanban boards.

Workflow and defects

- [ ] Explain statuses and transitions.
- [ ] Create and configure a workflow.
- [ ] Understand workflow schemes.
- [ ] Report bugs professionally.
- [ ] Explain the bug life cycle.
- [ ] Differentiate severity and priority.
- [ ] Retest, close, or reopen bugs.

Test management

- [ ] Write test scenarios and test cases.
- [ ] Record test execution results.
- [ ] Link failed test cases to bugs.
- [ ] Understand test plans and test suites.

Search and reporting

- [ ] Write basic JQL queries.

- [ ] Create and save filters.
- [ ] Share filters appropriately.
- [ ] Create a dashboard.
- [ ] Add and configure gadgets.
- [ ] Understand Jira reports.

Advanced Jira

- [ ] Understand permission schemes.
- [ ] Understand notification schemes.
- [ ] Configure issue type schemes.
- [ ] Understand screens and field configuration.
- [ ] Understand Jira automation.
- [ ] Understand GitHub and test-tool integrations.

81. Seven-day revision plan

Day	Topics
Day 1	Jira introduction, projects, issues, and issue types.
Day 2	Agile, Scrum, boards, backlog, and sprints.
Day 3	Workflows, statuses, and transitions.
Day 4	Bug reporting, bug life cycle, severity, and priority.
Day 5	Test cases, test execution, and issue linking.
Day 6	JQL, filters, dashboards, gadgets, and reports.
Day 7	Administration, automation, integrations, and interview questions.

82. Final understanding

Jira is not just a tool for creating bugs. It is a work-tracking and collaboration platform that connects requirements, development, testing, defect management, and project reporting. A tester should be able to understand a requirement, design and execute tests, report defects clearly, follow the bug workflow, retest fixes, and use filters and dashboards to communicate testing progress.

Focus especially on issue management, bug reporting, workflows, JQL, dashboards, Agile, and Scrum. These concepts form the foundation of practical Jira work and entry-level QA interviews.